

SCJP 5.0 Study Notes

Collections Framework (only Maps, Iterators do not extend java.io.Collection Interface)

Collection interface	add(E) addAll(Collection<E>) remove(Object) removeAll(Collection<?>) contains(Object) size() iterator() isEmpty() Object[] toArray() – returns Object array, must cast – e.g. String[] str = (String[]) myCollection.toArray(); T[] toArray(T[] a) – returns typecast array – syntax: String[] str = myCollection.toArray(new String[0]);
Map interface, SortedMap interface extends Map	put(keyObject, valueObject) – inserts key/value pair, will <i>overwrite</i> duplicate keyObject values keySet() – returns set of keys values() – returns collection of values get(Object), remove(Object), size(), clear() entrySet() – returns set of Map.Entry type (key/value pair) Map.Entry – a key/value pair, has getKey() and getValue() methods Maps care about unique identifiers There is no add() method; Don't have to override equals(Object) method to compile, but necessary to find objects
HashMap	
Hashtable	synchronized version of HashMap
TreeMap	Can have null values, but NO null keys
LinkedHashMap (extends HashMap)	maintains insertion order, slower updates, faster iteration than HashMap, order remains same if replacing duplicate key, can have null keys or values
Set interface, SortedSet interface extends Set	boolean add() – if element already exists, returns false and element not added Sets care about uniqueness
HashSet	
LinkedHashSet (extends HashSet)	fastest iteration through items without duplicates (a Set) that maintains insertion order
TreeSet<T>	new TreeSet(), new TreeSet(Collection<E>), new TreeSet<T>(Comparator<? super E>)
List interface	Lists care about the index
ArrayList	fastest iteration through items with duplicates
Vector	synchronized version of ArrayList; slower
LinkedList	addFirst(), getFirst() doubly linked, ideal for implementing a stack or maintaining an ordered sequence of objects when they are frequently inserted and removed from the middle of the sequence
Queue interface	peek() – returns highest priority entry without removing it poll() – returns highest priority entry and removes it offer() – adds entry
PriorityQueue	
Collections	Collections.sort(Collection) Collections.sort(Collection, Comparator) Collections.binarySearch(collectionName, pattern) – must be sorted first Collections.binarySearch(collectionName, pattern, Comparator) – must be sorted first Collections.reverse() – reverses order of elements in List, doesn't have to be sorted first Collections.reverseOrder() – returns Comparator that sorts in reverse, must be sorted first
Arrays	Arrays.sort(arrayToSort) Arrays.binarySearch(arrayName, pattern), Arrays.binarySearch(collectionName, pattern, Comparator) – returns int index Arrays.asList(arrayName) – returns List, changes to either returned list or original array will be reflected in the other
Comparable interface	int compareTo(SpecificObjectType) returns negative int if this < Obj2 0 if Objects are equal positive int if this > Obj2
Comparator class	int compare(SpecificObjectType1, SpecificObjectType2) – returns int with same meaning as above – substitute “SpecificObjectType1” for “this”
Iterator	boolean hasNext()

next() – **returns type Object** which must be cast if no generic type is specified for iterator, even if Collection that called iterator() method has generics, like List<Integer>

Can use generics syntax to avoid explicit casting, e.g. Iterator<Dog> = dogs.iterator();

Dates

java.util.Date	new Date() – date Object with current date new Date(long millis) – millis since Jan 1, 1970 getTime() – returns long setTime(long millis) internally stored as primitive long which is milliseconds between date and Jan 1, 1970
Calendar	Calendar.getInstance() – gets default Calendar (almost always GregorianCalendar) with current time Calendar.getInstance(Locale) add(intField, intAmount) – adds or decrements field. intAmount can be negative roll(intField, intAmount) – larger parts of date will not be incremented/decremented, only the field specified set(intField, intValue) get(intField) – e.g. calendarInstance.get(Calendar.MONTH); set() – various other set methods setTime(Date) getTime() – returns Date object, useful for passing to DateFormat setTimeInMillis(long millis) getTimeInMillis() – returns long compareTo(Calendar) NO new Calendar(), only instantiated using getInstance(), months are 0-based (January is 0)
java.text. NumberFormat	NumberFormat.getInstance(), NumberFormat.getInstance(Locale) – e.g. NumberFormat.getInstance(Locale.US); NumberFormat.getNumberInstance(), NumberFormat.getNumberInstance(Locale) – same as getInstance()? NumberFormat.getCurrencyInstance(), NumberFormat.getCurrencyInstance(Locale) format(long), format(double) – takes various numbers to be formatted, returns String parse(String) – takes a String (must be formatted correctly to locale), returns a Number – e.g. parse("\$5,075"); setParseIntegerOnly(boolean) – whether to format and display digits after decimal. getMaximumFractionDigits(), getMinimumFractionDigits – get number of decimal places setMaximumFractionDigits(int), setMinimumFractionDigits – set number of decimal places get/setMinimum/MaximumIntegerDigits() Should only instantiated using getInstance()
java.text. DateFormat	getInstance() – returns DateFormat with default Locale, sometimes SHORT style getDateInstance() (has defaults, sometimes MEDIUM style), getDateInstance(styleConstant), getDateInstance(styleConstant, Locale) parse(String) – converts String to a Date format(Date) – returns String with formatted Date <u>String format for each style</u> possible default format – 0/8/01 7:46 PM DateFormat.SHORT – 9/8/01 DateFormat.MEDIUM – Sep 8, 2001 DateFormat.LONG – September 8, 2001 DateFormat.FULL – Saturday, September 8, 2001 There is no format(Calendar) method. Calendar must convert to Date first(using yourCalendar.getTime()), then use format(Date) DateFormat's Locale can only be set at instantiation using getInstance() – NO new DateFormat()
java.util.Locale	Locale.getDefault() – returns default Locale new Locale(stringLanguage) new Locale(stringLanguage, stringCountry) – e.g. new Locale("it", "IT") setDefault(Locale) getDisplayLanguage() – returns String of Language getDisplayLanguage(Locale) – displays language using language of Locale passed in

getDisplayCountry() – returns String code
 getDisplayCountry(Locale) – displays language using language of Locale passed in)
 Locale.getAvailableLocales() – returns an array of Locales installed

used in Scanner, DateFormat, NumberFormat

String Stuff

String	new String(), new String(StringBuilder/Buffer) new String("stringLiteral") new String(ExistingStringObject) will create a new String object different than ExistingStringObject length() charAt(int) concat(String) equalsIgnoreCase(String) substring(intIndex) substring(startIntIndex, endIntIndexExclusive) replace(char/stringRegex, char/stringReplacement) replaceAll(char/stringRegex, char/stringReplacement) replaceFirst(stringRegex, stringReplacement) split(regex) – returns String array of tokens, good for just small amount of data split(regex, IntLengthOfArrayReturned) toLowerCase() toUpperCase() trim() – remove whitespace off ends toString() – returns a new String object indexOf(char/String) indexOf(char/String, startingIntIndex) lastIndexOf(char/String, startingIntIndex) lastIndexOf(char/String, startingIntIndex) contains(charSequence) matches(regex) – returns boolean indicating if regex matches string boolean startsWith(String) boolean startsWith(String, offsetIndex) String format("formattingString", args) – returns String formatted to specs String format(Locale, "formattingString", args) All methods return a String, do no alter original String NO insert() method (this is in StringBuilder/StringBuffer) NO length member variable (this is for array)
StringBuilder/ StringBuffer	append(XXX) – can append almost any char related type delete(intStartIndex, intEndIndexExclusive) deleteCharAt() insert(intOffset, String) reverse() toString() length() substring(startIndexInclusive) substring(startIndex, endIndexProbablyExclusive) lastIndexOf(String) lastIndexOf(String, startSearchIndex) replace(intStart, intEnd, String) setCharAt() StringBuffer is synchronized and slower. StringBuilder is not synchronized and faster equals() method is not overridden, so it compares references, not meaningful values – same as ==
java.util. Scanner	new Scanner(File), new Scanner(stringToSearchOrTokenize) useDelimiter(String) – can be regex useLocal(Locale) hasNext(), hasNextInt(), hasNextFloat(), hasNextXXX() – for all primitives EXCEPT char

	next(), nextInt(), nextFloat(), nextXxx – for all primitives EXCEPT char Scanner.close() – static method that should be used after done with file findInLine(regexString) – returns String found match() – returns MatchResult matchResult.groupCount() – method in MatchResult (not Scanner) matchResult.group(groupCountIndexInt) – method in MatchResult (not Scanner) Mostly used for tokenizing, but can also find regexes using findInLine() Default delimiter is whitespace
java.util.regex. Pattern	Pattern.compile(regex) – returns new Pattern matcher(CharSequence) – returns new Matcher split(regex) – returns String array of matching patterns found in the CharSequence Pattern.matches(regex, stringToSearch) – returns Boolean indicating if string matches regex /d digits /s whitespace /w word character (letters, digits, or underscores) [af] a or f [a-f] a, b, c, d, e, f [a-zA-D] A,a,B,b,C,c,D,d . any character + greedy one or more (e.g. /d+ means one or more digits) * greedy zero or more (e.g. filename/d*) ? greedy zero or more occurrences(e.g. /d?) ^ not (e.g. [^abc] excludes abc) String pattern = “\\d” creates string regex with \d. String pattern = “\\.” creates string regex with .
java.util.regex. Matcher	find() – returns boolean indicating a match’s existence, advances to next index that matches, left to right start() – returns starting index group() – returns string that matches <u>Probably Not on Exam</u> replaceAll(stringReplacement) replaceFirst(stringReplacement) appendReplacement(StringBuffer, stringReplacement) -- appendTail(StringBuffer) – to be used after appendReplacement
java.util. Formatter	new Formatter(), new Formatter(Appendable, Locale) – Appendable can be StringBuilder/StringBuffer, BufferedWriter, FileWriter, but NOT String format(“formattingString”, args) – returns formatted String

Serialization (package java.io, classes must implement Serializable to be serialized)

FileOutputStream	new FileOutputStream(stringFileName) new FileOutputStream(File)
ObjectOutputStream	defaultWriteObject() writeObject(Object) writeInt(int), writeFloat(float), writeXxx(xxx)
FileInputStream	new FileInputStream(stringFileName) new FileInputStream(File)
ObjectInputStream	defaultReadObject() readObject() – returns Object, must be cast before use readInt(), readFloat(), readXxx()

Serialization Notes

- For custom serialization, must have methods (EXACTLY as shown except parameter names):
 - private void writeObject(ObjectOutputStream oos)**, call oos.defaultWriteObject() before trying to do anything else in method
 - private void readObject(ObjectInputStream ois)**, call ois.defaultReadObject() before trying to do anything else in method
- If a superclass implements Serializable all subclasses do automatically
- If an object within a object to be serialized is not serializable, code will compile, but a RuntimeException will be thrown
- An object is serializable even if superclass is not. However, the first superclass in the hierarchy that does not implement Serializable must have a no-arg constructor (that will get called). If this is violated, readObject() will produce a java.io.InvalidClassException
- The (no-arg?) constructor of every non-serializable superclass will run when an object is deserialized. However, the deserialized objects’ constructor does not run when it is deserialized.
- Deserialization order is FIFO

- Only member (instance) variables are serialized
- Static and transient member variables are not serialized
- If a non-serializable element is present in a class you are trying to serialize, the code will compile but there will be a runtime exception upon attempt to serialize
- Filter Streams – another name for streams that wrap (get chained to) other streams

IO Stuff (java.io package)

File – can be file or directory	new File(pathNameAndFileString) new File(pathNameDirString, childPathNameString), new File(File pathname, fileORDirFileString) createNewFile() – creates file if doesn't yet exist, links to existing file, actually creates file on hard drive mkdir() exists() – returns boolean isDirectory(), isFile() – returns boolean delete() – can only delete directories if they are empty list(), list(FilenameFilter) – returns String array renameTo(File) – directory does not have to be empty to rename listFiles(), listFiles(FilenameFilter) – returns File array getName() – returns String getPath() canRead() NO method to change working directory
FileReader	read() No method for reading line at a time, almost always wrapped with BufferedReader
BufferedReader	read(), readLine() Usually wraps around FileReader, No method new BufferedReader(File)
FileWriter	new FileWriter(File) new FileWriter(File, appendToEndBoolean) – instead of overwriting new FileWriter(fileNameString) new FileWriter(fileNameString, appendToEndBoolean) – no overwrite write() flush() close() Almost always wrapped by (chained to) BufferedWriter
BufferedWriter	write() newline() flush() close() Usually wraps around FileWriter
PrintWriter	new PrintWriter(File), new PrintWriter(stringFileName), new PrintWriter(Writer), new PrintWriter(OutputStream) write() format()/printf() print()/println() flush(), close() Often wraps around BufferedWriter: PrintWriter out = new PrintWriter(new BufferedWriter(new FileWriter("foo.out"))); will buffer the PrintWriter's output to the file
FilenameFilter interface	accepts(directoryFile, nameString) – returns boolean indicating whether to file passes through filter or not used by File's list(FilenameFilter) and listFiles(FilenameFilter) methods
DataInputStream/ DataOutputStream	Not on exam despite what Sun objectives say– don't study

Other APIs in java.lang

Thread	new Thread(), new Thread(stringName), new Thread(runnableTarget), new Thread(runnableTarget, stringName) Thread.MIN_PRIORITY – usually 1 Thread.NORM_PRIORITY – usually 5 Thread.MAX_PRIORITY – usually 10 run() – can be overridden to make new thread, but not recommended start() – causes thread to move to alive state (available to run) isAlive() – returns boolean Thread.sleep(long millis) – throws InterruptedException join(), join(longMaxTimeToWaitInMillis) – throws InterruptedException Thread.yield() – usually shouldn't be used, but should return current thread back to runnable thread pool if other threads with same or higher priority exist setName(String) Thread.currentThread().getName() setPriority(int1-10) int getPriority() getState() boolean holdsLock()												
Object	wait(), wait(longMaxTimeToWaitInMillis), wait(longMaxTimeToWaitInMillis, intNanosMoreToWait) – throws InterruptedException notify() notifyAll() public boolean equals(mustBeObjectNotSubtype) public int hashCode() public String toString()												
java.lang.Math	Math.random() Math.round() Math.ceil() -- Returns the smallest (closest to negative infinity) double value that is greater than or equal to the argument and is equal to a mathematical integer. Math.floor() -- Returns the largest (closest to positive infinity) double value that is less than or equal to the argument and is equal to a mathematical integer. Math.round() Math.sqrt() Math.max(number, number) Math.min(number, number)												
java.lang.Enum	ordinal() – prints position (zero-based), similar to index for an array e.g. enumName.ordinal(); Enums in case statement should NOT use class prefix, but everywhere else should have prefix (e.g. Grade.A), unless enum is in the same class												
java.lang.System	System.out.println() System.out.print() System.out.format/printf(“string with %below”, argsToString) System.exit(intStatus) System.getenv() – returns Map of all environment variables System.getenv(stringEnvironmentVariable) – returns value for specified environment variable System.getProperty(stringKey) System.getProperties() – returns Property object (subclass of Hashtable) containing all properties System.setProperties(stringKey, stringValue) System.setProperties(PropertyObject) System.gc() – same as Runtime.getRuntime().gc() – requests (but may not obtain) garbage collection <u>For Formatting with format()/printf()</u> %[arg_index\$][flags][width][.precision]conversionCharacter -,+,0,,,(b,c,d,f,s,S <table border="0"> <tr> <td>- Left justify</td> <td>b boolean</td> </tr> <tr> <td>+ Include sign</td> <td>c char</td> </tr> <tr> <td>0 Pad with zeros</td> <td>d integer (digit)</td> </tr> <tr> <td>, Use locale-specific grouping separators</td> <td>f float</td> </tr> <tr> <td>(negatives in parentheses</td> <td>s string</td> </tr> <tr> <td></td> <td>S CAPITALIZE the string</td> </tr> </table> %illigitChar results in java.util.UnknownFormatConversionException	- Left justify	b boolean	+ Include sign	c char	0 Pad with zeros	d integer (digit)	, Use locale-specific grouping separators	f float	(negatives in parentheses	s string		S CAPITALIZE the string
- Left justify	b boolean												
+ Include sign	c char												
0 Pad with zeros	d integer (digit)												
, Use locale-specific grouping separators	f float												
(negatives in parentheses	s string												
	S CAPITALIZE the string												

Wrong format results in `IllegalFormatConversionException`
 Can have more arguments than conversions, but more conversions than arguments results in `java.util.MissingFormatArgumentException`
 You can't use an int literal argument for `%f`

Primitives, Wrappers, Assignments, Autoboxing

- `new XXX(String)`, `new XXX(primitive)` -- wrapper class constructors all take corresponding primitives or strings as arguments (e.g. `new Long("12")` or `new Long(12)`)
- `XXX.valueOf(String)`, `XXX.valueOf(String, intRadixBase)` – static method returns XXX wrapper type
- `parseXxx(String)` – static method returns primitive of type Xxx
- `xxxValue` -- converts wrapper to any primitive xxx type, truncates and does NOT round
- table on p. 233
- integer literals are always interpreted as int types by the JVM, number literals with decimals are interpreted as doubles
- byte – 8 bits, short – 16 bits, int – 32 bits, long – 64 bits, float – 32 bits, double – 64 bits, char – 16 bits (but can't fit into a short because it has a bigger positive range than short since there are no negatives)
- no static member can use a generic type parameter declared by the enclosing class
- Calling `run()` method will not create a separate thread, and `Thread.currentThread().getName()` of the thread in `run()` will be "main"
- Variables and classes can't be synchronized, but instance methods, static methods, and any code block within a method (enclose in `{}`) can be synchronized
- Conditional operators (`==`, `<`, `>`) have higher precedence than assignment operators, so "if (`b = x == y`)" can be a legal statement
- Wrappers with values over 127 are unique, so "`new Integer(130) == new Integer(130)`" returns false, but "`new Integer(120) == new Integer(120)`" returns true
- `WrapperClass.equals(AnyOtherClass) = false`, `String.equals(anyNonString) = false`, `StringBuffer.equals(AnyOtherObjectRef) = false` (`StringBuffer` does not override `equals` method)
- `0 % 4 = 0`
- `new Boolean("anyMessage");` will create a Boolean with false
- Trying to unbox a null will result in a `NullPointerException`
- if there are multiple `++x` incrementors in a compound expression, each is incremented before that part of the expression?
- For `Boolean bool = new Boolean("TRUE")`, in both `if (bool)` and `if (bool == true)`, `bool` is unboxed automatically
- `Number`, `StringBuffer/StringBuilder` DO NOT override `equals()` and `hashCode()` and inherit these from `Object` (so `.equals()` for different objects with the same values evaluates to false)
- For array instantiation, array size must be specified (INCORRECT – `in myArray[] = new int[]`)
- Vararg methods CAN be invoked with 0 or more arguments
- `Integer` can be boxed to `int` and vice-versa
- `Integer[]` CAN NOT be boxed to `int[]` and `int[]` CAN NOT be boxed into `Integer[]`
- ILLEGAL – `int[] values = new int[2] {1,2};`; LEGAL – `int[] values = new int[] {1,2};`
- To calculate `-5 % -2`, drop negatives, calculate remainder, then modify answer to have the same sign as operand on the left
- Casting floating point numbers to non-floats (e.g. `int I = (int) 23.4f`) truncates (does not round).
- In primitive casting, If value is too big, higher order bits (furthest left) are truncated.
- `(x=5)` returns 5, so `System.out.println(x=5)` will print "5"
- Wrappers can't widen to other Number types (`Integer` can NOT widen to `LONG`, `Short` can NOT widen to `INTEGER`)
- You can box and then widen (`int → Number`), but you can NOT widen and then box (`int → Long`)

Modifiers, Keywords, Declarations, and Assignments

- interface methods are public and abstract whether they say so or not – explicit declaration is optional, and using any other access modifiers will not compile
- interface methods can't be static or final, and variables can't be private or protected (but they can be static)
- interface variables are implicitly public, static, and final (they are constants), whether it is indicated or not – explicit declaration is optional, and using any other access modifiers will not compile
- Legal to do `String s = null; s += "";` if printed will print "null"
- When assigning array references to arrays, dimensions of array matter, number of indices in each array doesn't
 - e.g. `short[][] b = new short [4][4];`
 - `short b2 [][][] = new short [2][3][2][2];`
 - `b2 [1][0] = b;` //This is legal
- Even using static imports, a non-subclass object from a different package, can't access non-public static methods and variables
- Can't have a static method and non-static method with otherwise same signature in same class
- LEGAL – `public static int x = getValue();` (if `getValue` is static)
- `int[] myArray[]` is legal declaration for 2-D array
- Less Common Keywords: `strictfp`, `const`, `goto`, `native`, `transient`, `volatile`, `assert`, `enum`, `instanceof` (an operator NOT a method, `instanceOf` is OK)
- "native" only applies to methods and "strictfp" only applies to classes and methods
- "volatile" and "transient" modifiers only apply to instance variables.
- vararg parameters can take the specified type AND arrays of the specified type. e.g. `(String ...myVar)` can take several strings or a `String[]` array
- Trying to change the value of a final variable results in failed compilation
- JavaBeans standards – `camelCase` and `getXxx()`, `setXxx()`, `isXxx()`, `addXxx()`, `removeXxx()`

- final is the only legal modifier available for local variables
- There can only be 1 vararg in a parameter declaration, and it must come last in the parameter list of a method declaration
- In a vararg declaration, the space can be after the type, before the variable name, or nonexistent – e.g. (String... value), (String ...value), or (String...value)
- Access modifiers and other modifiers can be listed in any order (but before the return type in methods). “public static void” and “static public void” both work
- In a switch statement, the argument to the switch statement must be a byte, short, char, int, long, and enum. The arguments to the case statements must be constants: int-literals, static final int variables, or enums. They can be the result of arithmetic with the above as well (e.g. staticFinalVariable – 1). Any variation from this will not compile.
- “break outer;” means break out of the outer loop (not break to the outer loop)
- For an object declared final, the reference can NOT be changed, but the object it refers to CAN be changed

Inheritance, Polymorphism, Overloading, IS-A HAS-A stuff

- super only applies to calling overridden methods (and only non-private, non-static methods can be overridden, private methods are hidden from subclasses)
- Member variables are not overridden, but they can be legally shadowed, which is bad practice
- Polymorphism (different behavior depending on subclass type for superclass object reference) applies only to instance methods NOT static methods, and never to static or member variables.
- Static methods can be shadowed (redefined) but are not “overridden” by subclasses. Static (and static final) variables can also be shadowed (but variables are NEVER overridden). However, if method1() is a static shadowed method in car, in the code “Car c = new Honda(); c.method1()” will execute the method1 in the Car superclass, not the Honda subclass
- HAS-A relationships include static member variables
- Casts are not checked by the compiler, so illegal casts will compile fine but produce runtime exceptions
- The keyword “this” refers to the currently executing OBJECT, so it can be called from within static code
- Calling an overridden method always invokes the method “nearest” to the class instance
- For overloaded methods, autoboxing beats out varargs
- super keyword applies to subclasses accessing superclasses, not for inner classes. For inner classes to access outer class variables with the same names as inner class variables, the class name.this must be used (e.g. OuterClassName.this.variableName)
- implementing classes and subclasses do NOT have to explicitly declare exceptions of superclasses and implemented interfaces
- Overriding methods can NOT declare any new unchecked exceptions or broader checked exceptions than the overridden method, but they can declare any new runtime exceptions or errors (which are also unchecked but Throwable)

Encapsulation, coupling, cohesion

- Good encapsulation promotes loose coupling
- Coupling is not used to evaluate classes within the same inheritance tree
- Encapsulations limits the consequences of change
- Encapsulation allows corrections to a class with minimal impact to users of that class
- Encapsulation (and high cohesion) makes it easier to reuse classes
- Encapsulation results in better testing and higher reliability
- Cohesion refers to the number and diversity of tasks that a single unit is responsible for. A class with one task has high cohesion an any method not focused on a single task shows low cohesion
- Coupling is how linked and non-independent together two classes are. Loose coupling helps understand one class without studying the others and minimizes changes in one class when another class is changed.
- For method arguments, passing in specific, user-created objects requires tight coupling, since the two objects involved both need to know something about the object being passed in

Collections, equals(), and hashCode()

- In order to sort and array or collection, elements must be mutually comparable (same type) – trying to sort different objects will result in runtime exception
- The more unique the hashCode, the faster the retrieval
- the equals() method for an object must be at least as precise as the hashCode() method (equals() and hashCode() CAN use the exact same algorithm)
- .equals() should return false for an object of a different type (but compiles and runs without exceptions)
- HashSet and LinkedHashSet can have null elements, TreeSet can’t (it has to sort)
- HashMap and LinkedHashMap can have a null key and null values, TreeMap can have null values but NOT null keys, and Hashtable can have neither null keys or null values
- For non-generic TreeSet, putting objects of different types will compile fine, but this will produce a runtime error
- For any generics, trying to add the wrong type will not compile (e.g. trying to add a String when <Integer> was specified)
- When you try to add objects without a “compareTo” method to a Set, code will compile fine but a ClassCastException will be thrown at runtime (trying to cast object to a Comparable)
- It is LEGAL to assign generic type collections to raw type collections, e.g. Vector v = new Vector<Integer>()
- Trying to add an object that does not implement comparable to an sorted collection (Tree prefix) will not compile
- Natural ordering of Strings from low to high: space, numbers, all uppercase letters, underscore, all lowercase letters

Generics

- Legal generic method declaration: public static <X, Y extends X> boolean isPresent(X x, Y[] Y)

- Generic types (within “< >”) do not get upcast by the compiler, but base types (before the “< >”) using generics can be implicitly upcast (and downcast?)
- <? extends Animal> means the generic can be assigned an Animal or any subclass of Animal
- <? super Animal> means the generic can be assigned an Animal or any superclass of Animal
- List<?> is the same as (means) List<? extends Object>
- You cannot add objects to a Collection using generics <?> syntax. Compilation will fail if the generic type is not known at compile time. However, you can add to Collections using generics with <? extends MyObject> syntax
- In method declaration that can accept generics syntax, generics must be declared before the return type: public <T> void makelist(T t){}
- <T super Dog> not <? super Dog>
- It is VALID to assign non-generic classes to generic classes LinkedList<Integer> IList = new LinkedList() – gives warnings on compilation but runs without errors

Exceptions

- Exceptions that are not handled propagate (from main they propagate to the terminal) AFTER any finally
- Throwable is a class and NOT an interface, so it's LEGAL to write new Throwable()
- Programmers frequently throw IllegalStateException, NumberFormatException, and IllegalArgumentException, AssertionError, not the JVM. Know when to throw these.
- JVM usually throws NullPointerException, ArrayIndexOutOfBoundsException, ClassCastException, ExceptionInInitializerError, StackOverflowError, NoClassDefFoundError, and any other Error. Know when these are thrown.
- catch(Exception e) {} will catch all exceptions, including Runtime Exceptions that don't need to be caught to compile
- FileNotFoundException extends IOException (which is a RuntimeException)
- Constructors can throw exceptions

Command Line/Dev Environment Stuff

- For running programs from the command line, the classpath must include the directory to find the class file of the specified program (containing main) and the directories (root directory of any packages) for finding any class files referenced by the main program
- Can't call “super.super.someMethod()” – will not compile
- It is LEGAL to import only static methods with static imports – e.g. “import static java.util.Collections.sort”, then in code “sort(arrayList)”
- Static imports must include, either with a wildcard (*) or explicitly, the members that will be referenced. So “import static java.lang.Math” is not a valid static import, but “import static java.lang.Math.*;” or “import static java.lang.Math.random” are valid static imports
- jar files and “-classpath” (or “-cs”) can be used with both java and javac
- Command line calls to run a class (using java) must include package names (e.g. java -cp classes com.players.MusicPlayer)

Threads

- Thread name does NOT have to be unique
- To synchronize correctly, usually synchronize in method declaration, on “this”, or on a static object, but do NOT choose instance or local variables since each object has its own lock.
- A synchronized method can be overridden by a non-synchronized method and vice-versa
- Abstract methods can't be synchronized
- Objects have locks, Threads do NOT have locks
- Collections Framework interfaces CAN'T be serialized (probably no interface can be serialized?), but all of the implementing classes can
- Invoking start() multiple times on a thread will compile but throw an IllegalThreadStateException at runtime
- Only methods and blocks can have synchronized modifier, not variables or classes. However, you can synchronize on (the lock of) an object that may be a variable using synchronized(myObject) – synchronizes on lock of myObject
- synchronized(this) says to synchronize on current object and synchronized(MyClass.class) synchronized on the class (it's equivalent to synchronizing a static method)
- There are separate locks for the class (just one for class) and for the objects of the class (can be many for objects – 1 for each object)
- A thread must have a lock on the object on which the wait() method is to be invoked (but then it gives up the lock as soon as wait() is invoked)
- Call start() on a Thread instance, not a Runnable instance
- A lock is not released when a thread goes to sleep

Assertions

- It's considered OK to throw AssertionError explicitly (why? does this mean outside of an assertion)
- OK to use assertions to verify the arguments of private methods
- Assertions statements should not alter the value of any variable.
- the expression after the “:” in an assert statement must have a value, so a method call must have a return value, and after the colon can't be empty
- For assertions to execute, code needs to be both compiled AND run with assertions enabled

Inner Classes

- Inner classes can be abstract, and any abstract methods in abstract classes can be defined with a concrete anonymous subclass
- Correct static “inner” class instantiation syntax: new Outer.TestInner() – constructor for Outer will never run. If it's not a static inner class, then syntax needs to be new Outer().new TestInner()

Garbage Collection and finalize()

- Garbage Collection is performed by a daemon thread
- An object created and accessed locally may only be eligible for garbage collection when the method returns IF the object is not passed out of the method
- If an array of size 2 that refers to 2 objects is eligible for garbage collection, then 3 objects are actually eligible for gc – the array object and the 2 objects it references
- When an object is passed to an unseen method, we can't be sure the object is eligible for gc anywhere below the method-calling code, b/c the method could pass the object on to threads.
- The finalize() method can be overloaded, but only the no-arg finalize() method will be called by the JVM.
- The finalize() method is usually used to free up resources other than memory
- Live object references in a finalize() method can "resurrect" an object
- The finalize() method is guaranteed to run once and only once before the garbage collector deletes an object. However, there is no guarantee that the object will ever be garbage collected, so there is no guarantee that finalize() will run.

-
-
-

Enums

- myEnum.valueOf(String) – returns value of corresponding enum value
- yourEnum.values() lists all the possible values of yourEnum
- Enums can have instance/member variables, constructors, and methods
- Enums constructors can have a constant specific class body (a method that overrides one of the methods), but not any new methods? or any method calls
- In an enum declaration, multiple constant-specific methods for each enumerated type must be separated by commas and terminated by a semi-colon: e.g.


```
enum VALUE{ MAX {int returnIt(int x) {return 10}; },
              MIN {int returnIt{(int x) {return 1};};}
```
- enums can be imported using "import package.EnumClass.*", "import package.EnumClassor.EnumName", "import static" versions
- enum constructors can never be invoked directly from code

Simple Example

```
enum Month{NOV, DEC, JAN};    // The semi-colon at the end in this case is optional
```

More Complex

```
enum Month {
    NOV, DEC(11), JAN, FEB; // Any access mod -- SAME class, default/pub – own class
    Month(){}               // Must end in semicolon if more than just declarations
    Month(int index) {       // No-arg constructor only auto-created by JVM
        this.index = index; // if no other constructor provided – just like classes
    }
    int index;               // Initialized automatically, can have any access modifier
};                           // This semicolon is optional

for (Month month: Month.values()) { //notice values() method
    System.out.println("Day = " + month);
}

Month month = Month.DEC;
switch (month) {
    case DEC: System.out.printf("%s %d %b", "November", 264, true); //CAN'T use Month.DEC in case – won't compile
}
}
```

Really Complex

```
enum Month {
    NOV("November", 11), // Notice various constructor formats that all compile and run
    DEC("December"),
    JAN {int getIndex(){return 55;}}, // Notice overridden method here. CAN'T be new method – doesn't compile
    FEB(),
    MAR;

    Month(){}
    Month(String name, int index) {
        this.index = index;
    }
    Month(String name) {
        this.name = name;
    }
}
```

```

        protected int index;
        String name;
        int getIndex(){
            return this.index;
        }
        String getName(){
            return this.name;
        }
    };

```

Gotchas (very tricky stuff)

- Watch for faulty method returns, either returning types different than declared type or failing to return something, or returning something when void is declared, or not returning something if a return type was specified.
- Watch for methods parading as constructors – constructors do not have return values!
- If there is a method call to methods with the same name and with (String[] args) and (String... args), the code won't compile b/c it won't know which method to call
- Remember that after threadName.start() is declared, the thread may not enter the running state until all the code after it has been executed
- The class has a lock, and each object has a lock, and notify() and notifyAll() only notify threads waiting for the SAME LOCK
- Static methods can't override non-static methods and vice-versa – will not compile
- Watch out for statements that the compiler knows will never be reached (like after throwing an exception) – this code will not compile
- new Boolean("True") results in true, new Boolean("AnythingOtherThanCaseInsensitiveTrue") is false, boolean x = true or false (boolean x=True or boolean x=TRUE will not compile)
- in System.out.format/printf("%b", varX) will print true if varX has any value
- If Subclass is in a different package than Superclass, any protected variables in Superclass can only be reached from the main() in Subclass using an object reference to Subclass, not Superclass
- When wait() and notify() methods are called from non-synchronized context, code will compile, but IllegalMonitorStateException will be thrown at runtime
- HashMap and LinkedHashMap can have 1 null key and multiple null values, but TreeMap can't have any null keys (can have null values), and Hashtable can't have any null keys or values (will result in NullPointerException)
- Use of collectionType.toArray() and trying to use as array as specific types, without casting, when toArray() returns Objects.
- Static member variables are automatically initialized, final member variables must be initialized by the time the constructor finishes, and static final member variables must be assigned at time of declaration or in a static initialization block. Breaking these rules results in failed compilation
- Classes can have multiple static initialization blocks and instance initialization blocks, which I think run in the order listed
- Can't assume from listed code that a property value hasn't already been set for use of System.getProperty(stringKey, stringDefaultValue), so can't tell if property will be set to stringDefaultValue or not
- Watch out for trying to use elements of raw type (non-generic) collections without casting
- Watch out for static methods trying to make a reference to "this"
- Illegal to try to return literal 1.2 when return type is a float (the literal 1.2 is a double)
- Any methods implemented from an interface MUST be public since an interface's methods are always public
- Get suspicious when you see the protected access modifier – they are probably going to try to illegally access it from outside the package (you can only access it through inheritance, not through an instantiation of the object with the protected member)

Exam Tips

- After finishing, go back and check every question for:
 - calling non-static methods or using non-static variables in main
 - for any Thread question, all wait(), sleep(), and join() methods handle the InterruptedException
 - modifying strings but not having a reference to new modified string
- Get suspicious when there is a mix of raw type collections and generic type collections
- If question code contains packages and imports, watch out for probable errors using these.

Random notes